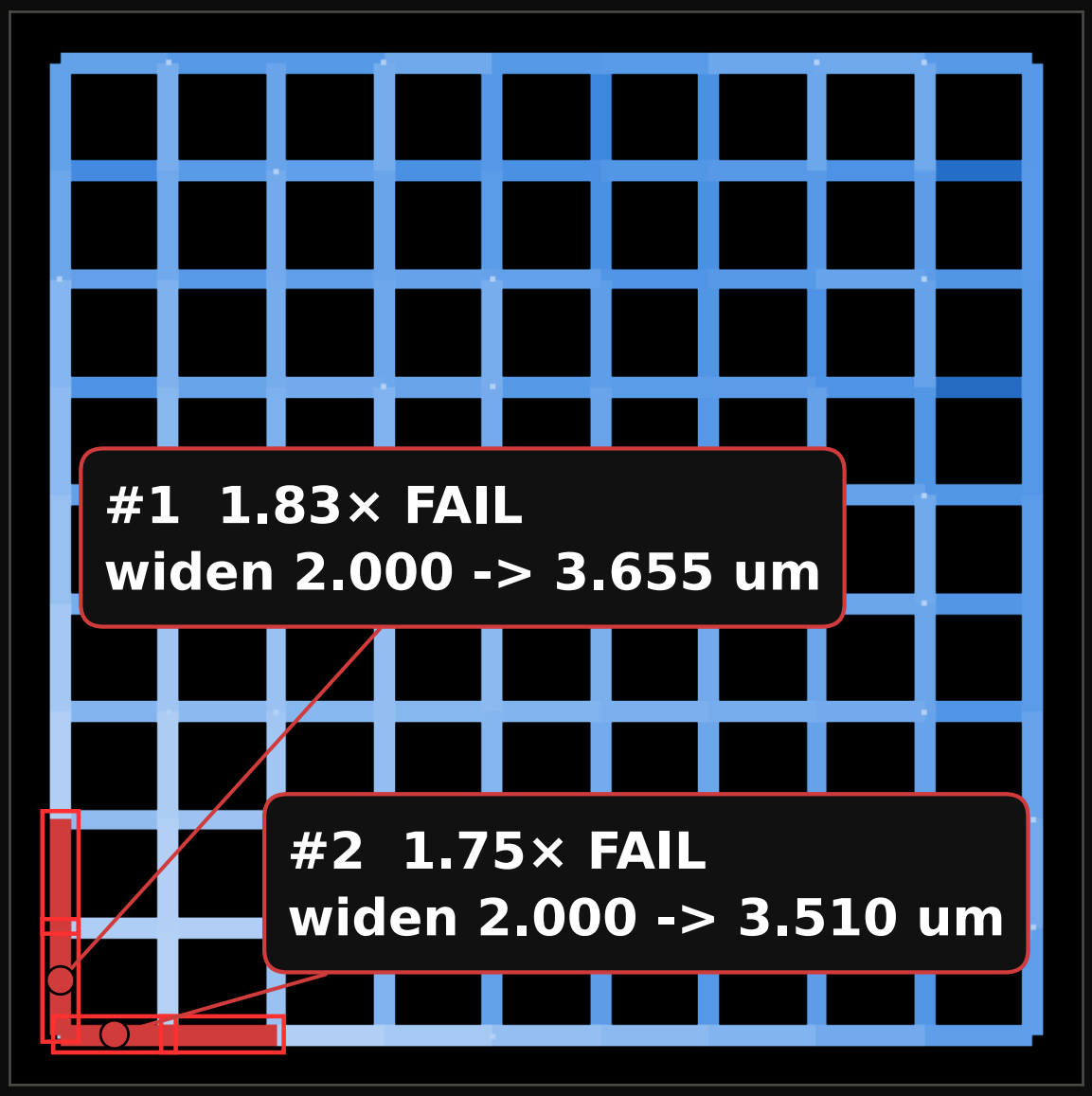
EM/IR screening your AI agent can run

Electromigration and IR drop on extracted analog layout: every segment checked, the fix in microns or cuts, a report a person can act on.



ok below 0.8×, blue by ratio
warn 0.8× to 1×
fail 1× and over, or not a number
violation marker

agent ▶ `emir next` ▶ `emir run` ▶ `emir report`

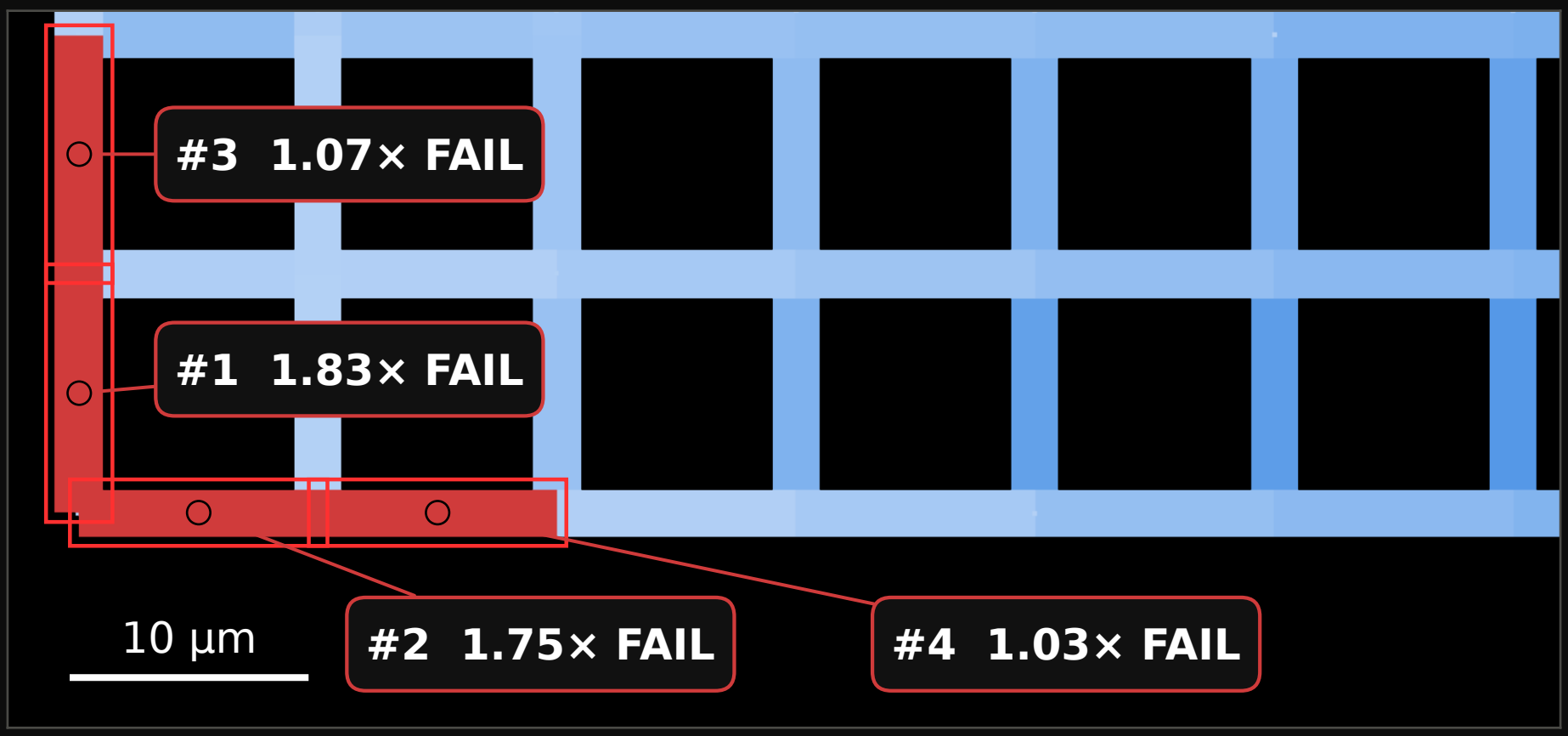
Screening, not sign-off.

---

examples/06_big_block: a synthetic 10 x 10 mesh, 32 inverters, Xyce currents through the self-consistent loop (4 simulations); invented limits. Swipe for how it works. Screening, not sign-off.

Every segment, against your limits, with the fix

EM ratio (avg, rms, peak) of each wire section and via cut, the IR drop, and what to change. Here: where the supply enters the mesh.



ok below 0.8×, blue by ratio
warn 0.8× to 1×
fail 1× and over, or not a number
violation marker

by hand	the run
Kirchhoff at the feed 3.655 + 3.509 + 0.231 = 7.395 mA #1 up, #2 right, the corner inverter	**7.395 mA** the supply's average current
#1, the strap up from the feed 3.655 mA / 2.0 µm / 1.0 mA/µm = 1.83× average current over width, over the limit	**1.83× FAIL** fix: widen 2.000 -> 3.655 um

examples/06_big_block, invented round-number limits; Xyce currents through the self-consistent loop (4 simulations). Screening, not sign-off.

Built to be driven by an AI agent

Every command answers in JSON, every failure carries a code, and the tool itself says what to run next.

21
commands, each with --format json

311
stable codes: branch on the code, not the text

30
JSON schemas (emir schema)

```
$ emir next --dspf loadrail.dspf --process …
mode: cc-grid  (the person's choice)
  1. emir selftest
  2. emir nets loadrail.dspf
  3. emir verify-sim --sim xyce
  4. emir simulate tb_cc.cir --sim xyce --dspf …
  5. emir check loadrail.dspf --process …
  6. emir run loadrail.dspf --process process.yaml …
  7. emir report --from emir_out/summary.json

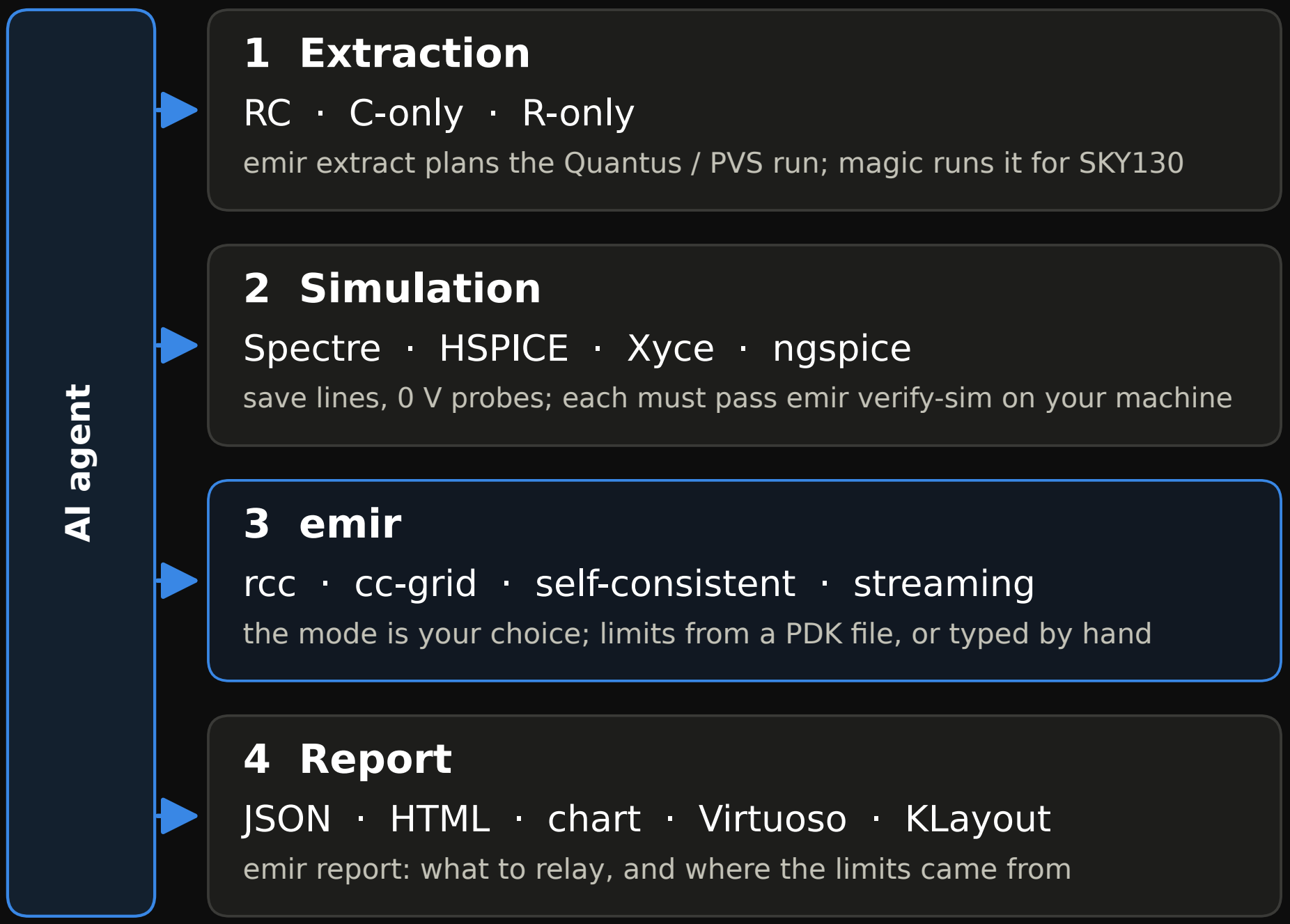
$ emir run loadrail.dspf --process process.yaml …
  overall verdict: FAIL -- 1 net(s): 1 FAIL, 0 …

$ emir report --from emir_out/summary.json
  Verdict: FAIL -- 1 net(s): 1 FAIL, 0 WARN, 0 …
  worst: Rc3 (Cont via) peak 1.042x
  fix: Cont: 1 -> 2 cuts (+1)
  IR drop VDD: 28.068 mV (from the feed)
```

The plan and its steps on examples/02_xyce, run as printed (scripted; cc-grid as the person chose; Xyce currents, the bundled illustrative limits); long lines shortened (…). Screening, not sign-off.

Your extraction and simulator in, one report out

The agent drives each stage with one emir command and relays the report.

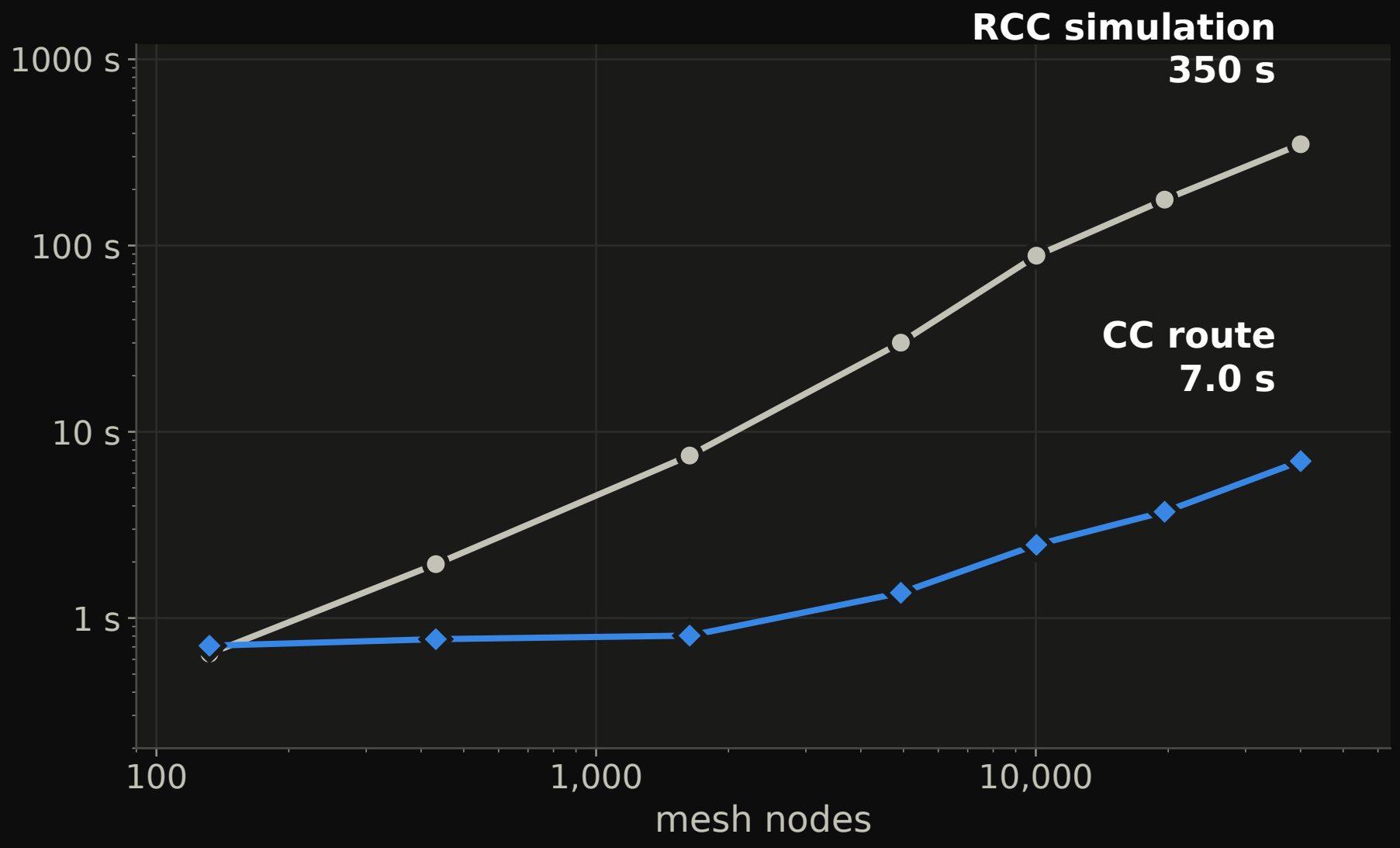


Big blocks: C-only simulation + R-only grid (cc-grid). Small blocks: RC simulation (rcc). You choose: the agent shows the measured trade-offs and asks.

Spectre, HSPICE, Quantus, PVS and Virtuoso need licences: none runs in this repository's tests, and each must be verified on site by your agent before you rely on it. Xyce, ngspice, KLayout and magic run in its examples.

Big blocks: skip the RCC simulation

Simulate C-only, push the device currents through the R-only grid. From 132 to 40,032 mesh nodes the RCC simulation went from 0.64 s to 350 s; the whole CC route stayed under 7 s.



50×

faster at 40,032 nodes:
RCC simulation against the CC route

1,298 → 1.4

MiB of simulation result,
RCC against C-only

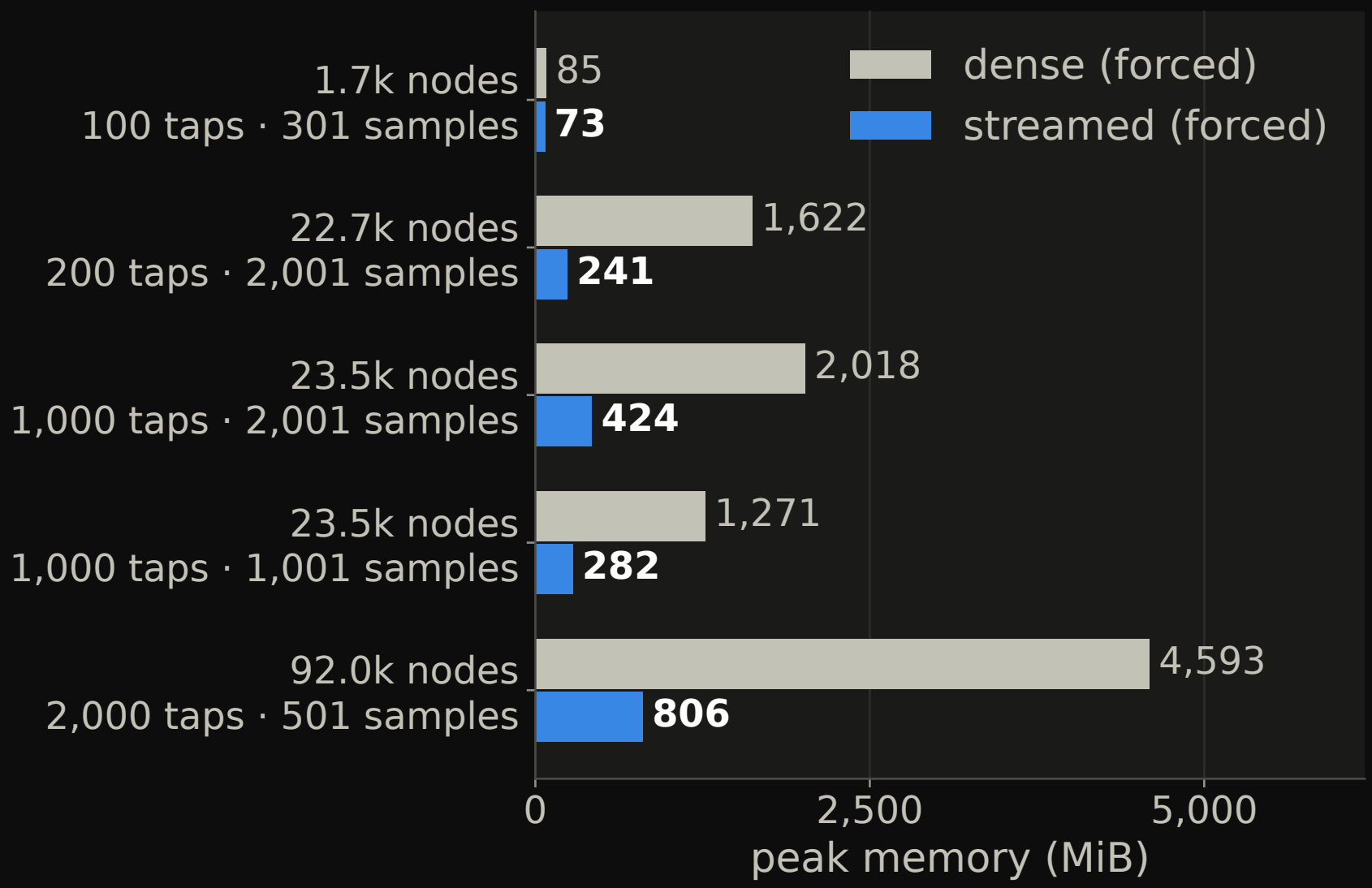
Xyce, synthetic, serial: examples/06_big_block/results/sweep.csv, the same 32 inverters on a growing mesh; CC route = C-only simulation + emir cc-grid. Trends, not absolute times. Screening, not sign-off.

Big grids stream in bounded memory

A 92,001-node grid with 2,000 device taps would need a 2.70 GiB dense transfer matrix; EMIR streams it instead, with the default settings:

11.7 s
wall time, defaults

804 MiB
peak memory, defaults



Synthetic meshes, emir run single-threaded (examples/06_big_block/results/stream_bench.csv). The bars force each path; dense against streamed: the same currents to 7e-15. Screening, not sign-off.

How close is cc-grid to RCC?

cc-grid minus RCC, the branch that decides the verdict, over four mesh sizes. Plus is conservative.

avg **−0.15 to −0.12 %**

every branch over 1 % of the largest: −0.21 to −0.02 %

rms **+6.83 to +13.68 %**

every branch over 1 % of the largest: +4.1 to +13.8 %

peak **+15.28 to +24.29 %**

every branch over 1 % of the largest: −19.0 to +37.8 %

IR drop **−0.15 to −0.13 %**

the worst node of the net

dynamic IR **+14.49 to +16.66 %**

open loop, conservative on every mesh measured

Open loop at 4.1 % droop (8x the devices)

avg +1.34 %, rms +38.2 %, peak +79.9 %: the error grows with the droop

--self-consistent on a 128-inverter mesh at 4.1 % droop: page 9.

Read cc-grid avg and IR as ±2 %; confirm a fix that a secondary branch's rms or peak decides with an rcc run of that sub-block. Above about 2 % droop, report the open and closed loop side by side.

Xyce, synthetic, serial: examples/06_big_block/results/sweep.csv (32 inverters, N = 10 to 70; 256 at N = 70, the 4.1 % case). The guard band is the docs' recommendation from these numbers. Screening, not sign-off.

Chip-level EM/IR, block by block

Characterise each block once, screen the top with the blocks black-boxed, then re-check the blocks with the top's pin voltages.

1 emir abstract

Each block once: its pin currents, port conductance and internal IR drop, in one JSON file.

2 emir run --abstract

The top with the blocks black-boxed: equal to a flat run of the same chip to 5e-13.

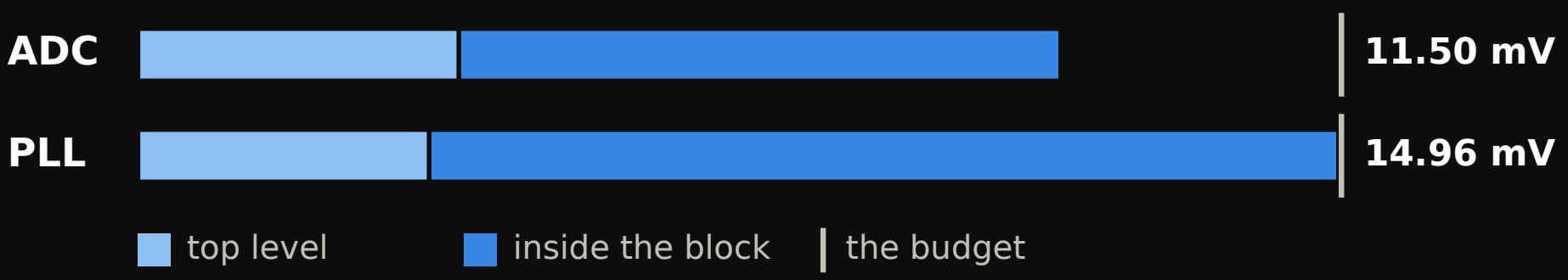
3 budgets

IR per block = top + inside: ADC 4.00 + 7.50 = 11.50 mV, within 15 mV.

4 back-annotation

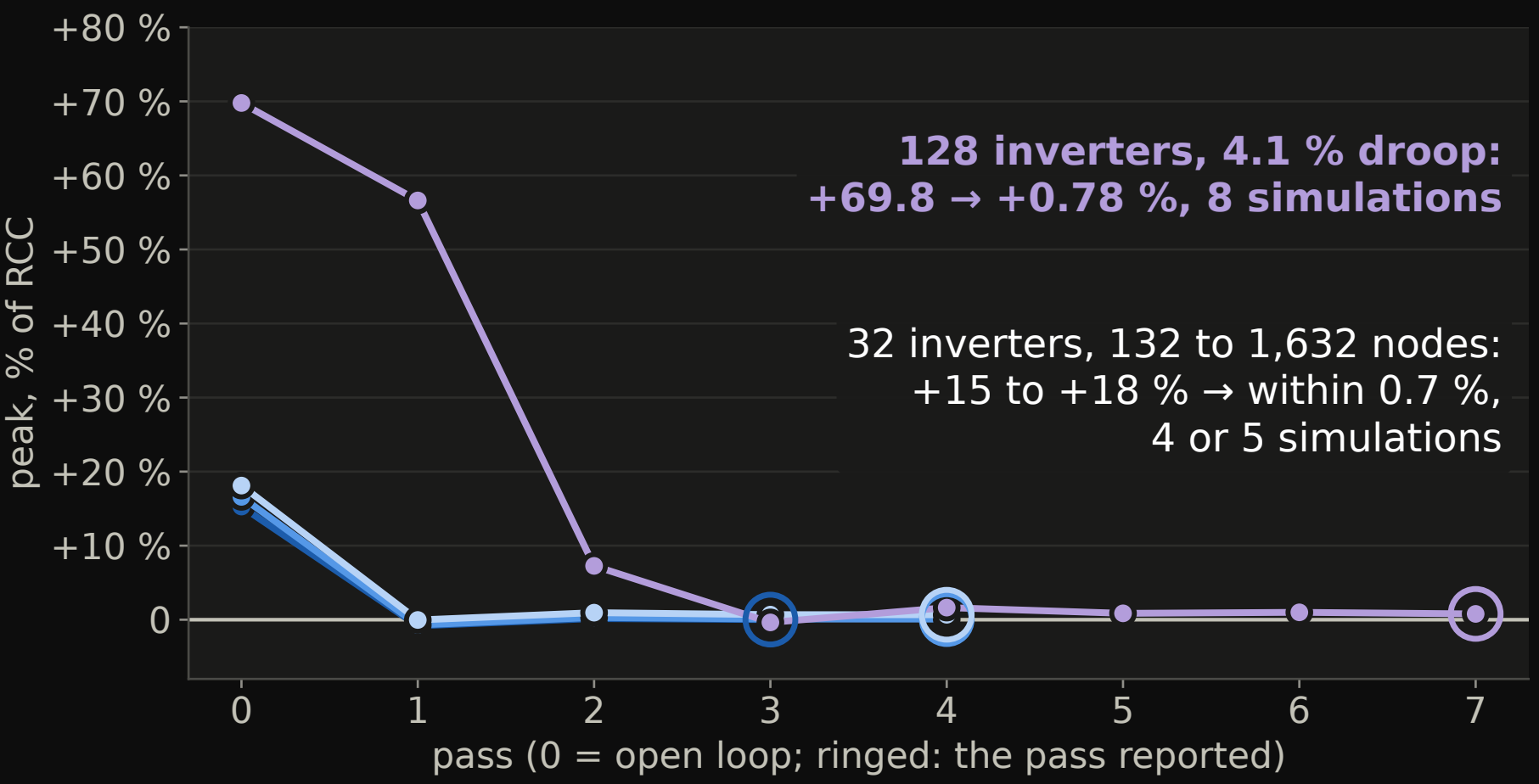
A PLL segment read 0.952× (WARN) with its pins held ideal, 1.005× (FAIL) with the top's voltages.

IR budget per block, 15 mV (PASS)



Feed each pin its own droop, then simulate again

--self-consistent re-simulates the CC testbench with every device pin fed its own droop waveform, until the droop the currents produce is the droop they were fed.



What it leaves: the grid's own capacitance

Scored on the R-only grid, minor branches read within 0.77 % of RCC on 132 nodes, up to 11.3 % off on 1,632, and within 0.2 % of RCC without the grid's capacitors. Accurate, not conservative.

On grids this small the loop costs more than RCC: 31.3 s against a 1.9 s RCC simulation at 4.1 % droop. The pay-off on a big block is an extrapolation, not a measurement.

Xyce, synthetic, serial: examples/06_big_block/results/selfconsist_waveform.csv, the governing branch of meshes with 2 fF of grid capacitance a node. Screening, not sign-off.

Point your agent at the repository

It reads AGENTS.md and starts by proving the install.

1 `emir selftest`
proves the install, offline, in seconds

2 `emir next --dspf ... --process ...`
the plan for what you have; you choose the mode

3 `emir run ... --mode ... --summary-out s.json`
EM and IR on every net

4 `emir report --from s.json`
what the agent tells you

Agents	Claude Code, Codex, any MCP client
Simulators	Spectre, HSPICE, Xyce, ngspice (verify each first)
Limits	pdks/ for open PDKs, or typed from your manual
Viewers	chart, HTML, Virtuoso, KLayout

Screening, not sign-off.

Foundry-qualified EM/IR remains the sign-off flow; EMIR says where to look first, and what it did not check. A research project, with no guarantee of accuracy: test and verify its results.

Every number in these pages comes from a run in the repository (scripts/make_figures.py).